SQL FOUNDATIONS · QUERY THINKING

SQL 01 · Introduction to SQL

SQL is how analysts ask structured questions of tables and turn rows into evidence.

Qode Clarity · SQL Job-Ready Track

MENTAL MODEL

How to Think About Introduction to SQL

Start with the shape of the problem before choosing tools.

- Start with SCQA before writing syntax: Situation, Complication, Question, Answer.
- Identify the table, the row grain, and the columns you need.
- Read SQL as a pipeline: FROM the data, WHERE filters rows, SELECT returns columns.
- Learn the common SQL core first, then notice dialect differences by database engine.
- Practise with small tables first so mistakes are visible.
- Every useful SQL file should be runnable, commented, and tied to a real question.

Plain-English Anchor

SQL is how analysts ask structured questions of tables and turn rows into evidence.

The Terms Learners Must Own

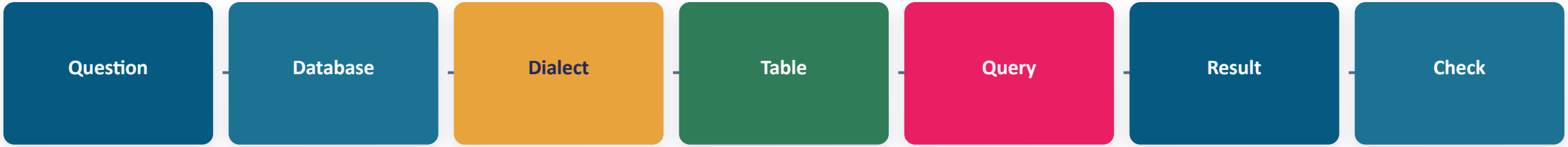
These are the terms they should be able to explain without reading notes.

Concept	Meaning	Example / use
SCQA	A way to unpack business prompts	Situation, Complication, Question, Answer
Database	A container for related tables	Retail database with customers, orders, products
Table	Rows and columns about one kind of thing	orders table, products table
Row	One record at the table's grain	one order, one customer, one payment
Column	One field stored for every row	order_date, amount, region
Query	A written question for the database	show sales by region
Result set	The table SQL returns after running	the answer you inspect/export
Schema	The structure and rules of tables	columns, keys, data types
Primary key	The ID that identifies one row	customer_id
Foreign key	The ID that connects tables	orders.customer_id
SQL dialect	A database-specific version of SQL	PostgreSQL, SQL Server, SQLite, BigQuery

WORKFLOW

The Architecture Flow

A good Azure answer names the path data takes and why each step exists.



DECISION GUIDE

How to Choose the Right Pattern

These decisions turn tool knowledge into engineering judgement.

- Use SCQA to turn long scenario questions into one answerable SQL question.
- Use SELECT * only for a first look; name columns in real work.
- Filter early with WHERE when you only need some rows.
- Always know which SQL dialect you are using before copying syntax.
- PostgreSQL is common in backend/data engineering; strong types, rich functions, and open-source tooling.
- SQL Server uses T-SQL; expect TOP, square brackets, GETDATE(), and Microsoft ecosystem integration.
- SQLite is lightweight and file-based; great for practice, but looser with data types than production databases.
- BigQuery is a cloud warehouse dialect; expect backticked table names, Standard SQL, and cost-aware querying.
- Dialect differences often appear in dates, string functions, LIMIT/TOP, data types, and quoting rules.
- Comment the business question above every saved query.
- Run tiny checks: row count, null count, and one manual sample before trusting output.
- Save queries in small files by purpose, not as one giant scratchpad.

FAILURE MODES

Common Mistakes to Avoid

These are the mistakes that make demos fail in production.

- Writing syntax before understanding the question
- Using SELECT * in final evidence
- Confusing a row count with a business metric
- Ignoring NULL values and duplicate IDs
- Copying PostgreSQL syntax into SQL Server, SQLite, or BigQuery without checking the dialect
- Saving queries with no comments or expected result
- Changing data before learning how SELECT works

WORKED SCENARIO

Retail Orders: SCQA First SQL Story

Walk through this out loud before setting learners loose on the practice prompt.

- Situation: Meridian Retail has daily order rows from several regions and products.
- Complication: Managers see revenue movement, but they do not know which region and product combination is driving it.
- Question: Which products sold this week, by region, and where should management look first?
- Answer shape: filter this week's rows, group by region and product, sum amount, sort highest first.
- SQL shape: FROM orders → WHERE order_date in week → GROUP BY region, product → ORDER BY revenue DESC.
- Check: inspect row count, one manual product total, and whether NULL regions exist before sharing.

QUESTION BREAKDOWN

Real SQL Questions: How to Pick Syntax

Translate the business question into grain, filter, grouping, sorting, and output.

Business question	Breakdown / pseudocode	Likely SQL syntax
A retail manager says weekly sales are busy but unclear. Which products sold this week, and what should they review first?	SCQA: busy week → too many rows → which products? → filter week, select product/order fields	SELECT columns FROM orders WHERE order_date >= start AND order_date < next_start
The finance lead needs Monday's regional revenue summary before a pricing meeting. Which regions produced the most revenue?	SCQA: regional sales exist → no ranked view → which region leads? → group region, sum amount	SELECT region, SUM(amount) FROM orders GROUP BY region ORDER BY SUM(amount) DESC
Customer success wants to contact customers at risk of churn. Which customers have not ordered in the last 90 days?	SCQA: customer list + orders → inactivity hidden → who is stale? → left join, max order date, filter	LEFT JOIN + MAX(order_date) + HAVING last_order < cutoff OR last_order IS NULL
Marketing cannot send a campaign because some customer records have unusable emails. Which rows need fixing?	SCQA: campaign list exists → emails may be blank → which rows fail? → filter NULL or blank email	WHERE email IS NULL OR TRIM(email) = ''
Operations can restock only five products this morning. Which products have the highest quantity sold?	SCQA: stock decision needed → limited restock slots → top 5 products? → group product, sum quantity	GROUP BY product ORDER BY SUM(quantity) DESC LIMIT 5

QUESTION BREAKDOWN

5 More SQL Question Patterns

The same breakdown habit works across quality checks, joins, time analysis, and duplicates.

Business question	Breakdown / pseudocode	Likely SQL syntax
The loyalty team wants to segment customers by order frequency, including customers with zero orders. How many orders did each customer place?	SCQA: customer base exists → zero-order customers matter → count per customer → left join and count	LEFT JOIN orders; COUNT(order_id); GROUP BY customer_id, customer_name
A fraud analyst is doing a first pass on high-value transactions. Which orders are above 100 pounds?	SCQA: all orders exist → high values need review → which rows cross threshold? → fixed WHERE filter	WHERE amount > 100
The category manager wants to know which categories are improving over time. Which product categories are growing month by month?	SCQA: category sales exist → trend is hidden → growth by month? → month/category aggregate first	date/month grouping + SUM(amount), then use LAG in later lessons
HR imported staff records from two systems and suspects duplicates. Which staff records are duplicated?	SCQA: merged staff list exists → duplicates inflate headcount → which identity repeats? → group key, count	GROUP BY email/name/date_of_birth HAVING COUNT(*) > 1
A store operations lead expects every branch to report today. Which stores have no sales today?	SCQA: all stores should report → some may be missing → which stores have no matching sales? → anti-join	stores LEFT JOIN sales_today WHERE sales_today.store_id IS NULL

4 Take-Home SQL Challenges

Learners should write the query, run it, verify one row manually, and explain the result.

Challenge	Breakdown / pseudocode	Evidence to submit
A finance analyst needs to review unusually high transactions before approving daily revenue. Find high-value orders.	SCQA then SQL: define high value as amount > 100; filter rows; sort highest first	`sql/week-01/high-value-orders.sql` with query, expected count, and one checked row
A regional director asks which region should receive extra support next week. Build revenue by region.	SCQA then SQL: group order rows by region; sum amount; rank high to low	`sql/week-01/revenue-by-region.sql` plus one sentence naming the strongest region
Marketing is preparing a campaign and needs a clean contact list. Find missing or blank customer emails.	SCQA then SQL: identify bad email rules; filter NULL and blank strings	`sql/week-01/missing-emails.sql` with notes on NULL vs blank string
Customer success wants to identify signed-up users who never purchased. Find customers with no orders.	SCQA then SQL: keep all customers; left join orders; filter unmatched order rows	`sql/week-01/customers-no-orders.sql` with a short explanation of why INNER JOIN would fail

PRACTICE PROMPT

Create a tiny orders table, insert 6 rows, then write 5 SELECT queries that answer plain-English questions.

Deliverable: sql/week-01/intro-to-sql.sql with CREATE TABLE, INSERT rows, 5 commented SELECT queries, and one paragraph explaining SQL dialects and how to practise SQL well.

Use SOLVE: Situation → Observe → Look for tools → Verify → Execute